Pen and Paper Task 1 (5 points)

Given the following documents:

• Document 1: water nice

• Document 2: water

• Document 3: water

• Document 4: water

• Document 5: sun nice

• Document 6: sun is nice

• Document 7: water sun

• Document 8: sun

• Document 9: sun

Subtask 1.1

Build the full inverted index (without positional information) for these documents. Do not perform normalization of terms.

Answer:

Steps to build inverted index:

1. Tokenization and Preprocessing : Skipped (no normalization)
2. Generating postings :

Term	Document ID
water	1
nice	1
water	2
water	3
water	4
sun	5
nice	5
sun	6
is	6
nice	6
water	7
sun	7
sun	8
sun	9

3. Sort Postings alphabetically

Term	Document ID
is	6
nice	1
nice	5
nice	6
sun	5
sun	6
sun	7
sun	8
sun	9
water	1
water	2
water	3
water	4
water	7

4. Create Postings list

Term	Frequency	Postings List
is	1	[6]
nice	3	[1] -> [5] -> [6]
sun	5	[5] -> [6] -> [7] -> [8] -> [9]
water	5	[1] -> [2] -> [3] -> [4] -> [7]

Subtask 1.2

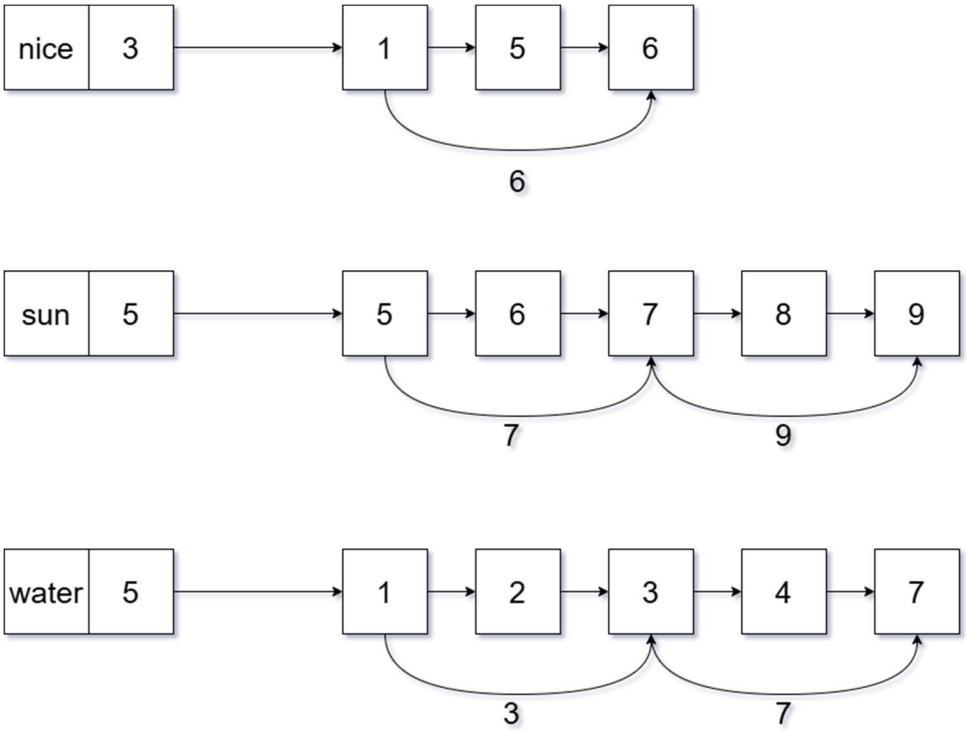
Add skip pointers to the index from the previous task. Provide an example query which demonstrates the usefulness of skip pointers for your index and explain why this query can be answered in a more efficient way with skip pointers than without

Answer:

Skip Pointers : For adding skip pointers to a postings list of length P, we use approximately √P evenly spaced skip pointers.

Term	Postings List	√P	Skip pointer
is	[6]	√1 = 1	Posting List only has 1 element, no skip needed
nice	[1] -> [5] -> [6]	√3= 1.73	1 skip pointer from [1] to [6]
sun	[5] -> [6] -> [7] -> [8] -> [9]	√5= 2.23	2 skip pointers. From [5] to [7] and from [7] to [9]
water	[1] -> [2] -> [3] -> [4] -> [7]	√5= 2.23	2 skip pointers. From [1] to [3] and from [4] to [7]

Positions of skip pointers:



Now, to test the effectiveness of these pointers lets consider the example query: “sun and water”

[not including nice as it is a short postings list and merging a longer list with a shorter one ends when the shorter list ends]

Without skip pointers the querying process would look like:

Current position in sun posting list	Current position in water posting list	Comparision	Result	Number of steps
5	1	5>1	Move one position forward in water list	1
5	2	5>2	Move one position forward in water list	2
5	3	5>3	Move one position forward in water list	3
5	4	5>4	Move one position forward in water list	4
5	7	5<7	Move one position forward in sun list	5
6	7	6<7	Move one position forward in sun list	6
7	7	7=7	Terms found in both documents, add document id to output Output of query =[7]	**7**
We have reached the end of the water posting list, so output of query “sun and water” has only 1 document [Document 7]				

With skip pointers the querying process is:

Current position in sun posting list	Current position in water posting list	Comparision	Skip Comparision	Result	Number of steps
5	1	5>1	If 1 skips to 3 5>3 ie skip is possible	Skip position forward in water list from 1 to 3	1
5	3	5>3	If 3 skips to 7 5<7 ie skip is not possible	Move one position forward in water list	2
5	4	5>4		Move one position forward in water list	3
5	7	5<7	If 5 skips to 7 7=7 ie skip is possible	Skip position forward in sun list from 5 to 7	4
7	7	7=7		Terms found in both documents, add document id to output Output of query =[7]	**5**
We have reached the end of the water posting list, so output of query "sun and water" has only 1 document [Document 7]					

Here we see that although the output of the query is the same in both cases, the number of steps to query the index without skip pointers[7] is greater than the number of steps to query with skip pointers [5]. This implies that the querying process is more efficient if we build the skip pointers first.